

```
In [1]: pip install pandas openpyxl mysql-connector-python
```

```
Requirement already satisfied: pandas in c:\users\bit\appdata\local\programs\python\python313\lib\site-packages (2.3.3)
Requirement already satisfied: openpyxl in c:\users\bit\appdata\local\programs\python\python313\lib\site-packages (3.1.5)
Requirement already satisfied: mysql-connector-python in c:\users\bit\appdata\local\programs\python\python313\lib\site-packages (9.6.0)
Requirement already satisfied: numpy>=1.26.0 in c:\users\bit\appdata\local\programs\python\python313\lib\site-packages (from pandas) (2.4.0)
Requirement already satisfied: python-dateutil>=2.8.2 in c:\users\bit\appdata\local\programs\python\python313\lib\site-packages (from pandas) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in c:\users\bit\appdata\local\programs\python\python313\lib\site-packages (from pandas) (2025.2)
Requirement already satisfied: tzdata>=2022.7 in c:\users\bit\appdata\local\programs\python\python313\lib\site-packages (from pandas) (2025.3)
Requirement already satisfied: et-xmlfile in c:\users\bit\appdata\local\programs\python\python313\lib\site-packages (from openpyxl) (2.0.0)
Requirement already satisfied: six>=1.5 in c:\users\bit\appdata\local\programs\python\python313\lib\site-packages (from python-dateutil>=2.8.2->pandas) (1.17.0)
Note: you may need to restart the kernel to use updated packages.
```

```
[notice] A new release of pip is available: 25.3 -> 26.1.2
```

```
[notice] To update, run: python.exe -m pip install --upgrade pip
```

```
In [2]: import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
```

```
In [3]: # Load the data
```

```
import pandas as pd
import mysql.connector

conn = mysql.connector.connect(
    host="localhost",
    user="root",
    password="password",
    database="project_1"
)

df = pd.read_sql("SELECT * FROM ecommerce_sales_data", conn)
df2 = pd.read_sql("SELECT * FROM customer_summary", conn)

df.to_excel("ecommerce_sales_data.xlsx", index=False)
df2.to_excel("customer_features.xlsx", index=False)
```

```
C:\Users\BIT\AppData\Local\Temp\ipykernel_9700\1337639063.py:13: UserWarning: pandas only supports SQLAlchemy connectable (engine/connection) or database string URI or sqlite3 DBAPI2 connection. Other DBAPI2 objects are not tested. Please consider using SQLAlchemy.
```

```
df = pd.read_sql("SELECT * FROM ecommerce_sales_data", conn)
```

```
C:\Users\BIT\AppData\Local\Temp\ipykernel_9700\1337639063.py:14: UserWarning: pandas only supports SQLAlchemy connectable (engine/connection) or database string URI or sqlite3 DBAPI2 connection. Other DBAPI2 objects are not tested. Please consider using SQLAlchemy.
```

```
df2 = pd.read_sql("SELECT * FROM customer_summary", conn)
```

```
In [4]: df.head()
```



```
In [13]: # standardization -- limiting the range of data into mean = 0 and standard deviation = 1
# Normalization -- limiting the range of data between 0 and 1
```

```
X_normal_scaled = df2_normal.copy()

scaler_normal = StandardScaler()
X_normal_scaled[features] = scaler_normal.fit_transform(df2_normal[features])

X_pre_scaled = df2_pre_registration.copy()

scaler_pre = StandardScaler()
X_pre_scaled[features] = scaler_pre.fit_transform(df2_pre_registration[features])
```

```
In [14]: # Train the Model
```

```
kmeans_normal = KMeans(n_clusters=3 , random_state =42)

df2_normal['cluster'] = kmeans_normal.fit_predict(X_normal_scaled[features])
```

```
In [15]: kmeans_pre = KMeans(n_clusters=3, random_state =42)
```

```
df2_pre_registration['cluster'] = kmeans_pre.fit_predict(X_pre_scaled[features])
```

```
In [16]: # summary of the model
```

```
cluster_summary = (
    df2_normal.groupby('cluster')[features]
    .mean()
    .round(2)
)

cluster_summary
```

```
Out[16]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th {
vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th>
<th>registered_to_order_duration</th> <th>first_to_last_order_duration</th> <th>total_unit</th>
<th>total_spent</th> <th>cluster</th> <th></th> <th></th> <th></th> </thead>
<tbody> <th>0</th> <th>1</th> <th>2</th> </tbody> <tbody></tbody>

114.36  128.85    2.90    129.40

121.50  387.25  588.25  16756.84

282.22  379.13    4.10    189.95
```

```
In [17]: cluster_summary = (
    df2_pre_registration.groupby('cluster')[features]
    .mean()
    .round(2)
)

cluster_summary
```

```
Out[17]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th {
vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th>
<th>registered_to_order_duration</th> <th>first_to_last_order_duration</th> <th>total_unit</th>
<th>total_spent</th> <th>cluster</th> <th></th> <th></th> <th></th> <th></th> </thead>
<tbody> <th>0</th> <th>1</th> <th>2</th> </tbody> <tbody></tbody>
-173.40    26.20     1.85     85.69
-14.00    329.00   586.00  16692.58
-67.02    240.49     4.24    184.31
```

```
In [18]: # Mapping

cluster_labels = {
    0: "Normal but low Engagement",
    2: "Extreme High Volume",
    1: "Loyal, High-Value Customer"
}
df2_normal["segment"] = df2_normal['cluster'].map(cluster_labels)

df2_normal.head()
```

```
Out[18]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th {
vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th>
<th>customer_id</th> <th>registered_to_order_duration</th> <th>first_to_last_order_duration</th>
<th>total_unit</th> <th>total_spent</th> <th>cluster</th> <th>segment</th> </thead> <tbody>
<th>0</th> <th>1</th> <th>2</th> <th>3</th> <th>4</th> </tbody> <tbody></tbody>
CUST_03731    562    692    6.0   295.254288    2      Extreme High Volume
CUST_03071    466    272    3.0   254.375567    2      Extreme High Volume
CUST_00908    263     0    1.0    97.368058    0    Normal but low Engagement
CUST_02852    417    542    2.0    88.990766    2      Extreme High Volume
CUST_00510    173    406    2.0   257.549943    2      Extreme High Volume
```

```
In [19]: cluster_labels = {
    0: "Normal but low Engagement",
    2: "Extreme High Volume",
    1: "Loyal, High-Value Customer"
}
df2_pre_registration["segment"] = df2_pre_registration['cluster'].map(cluster_labels)

df2_pre_registration.head()
```

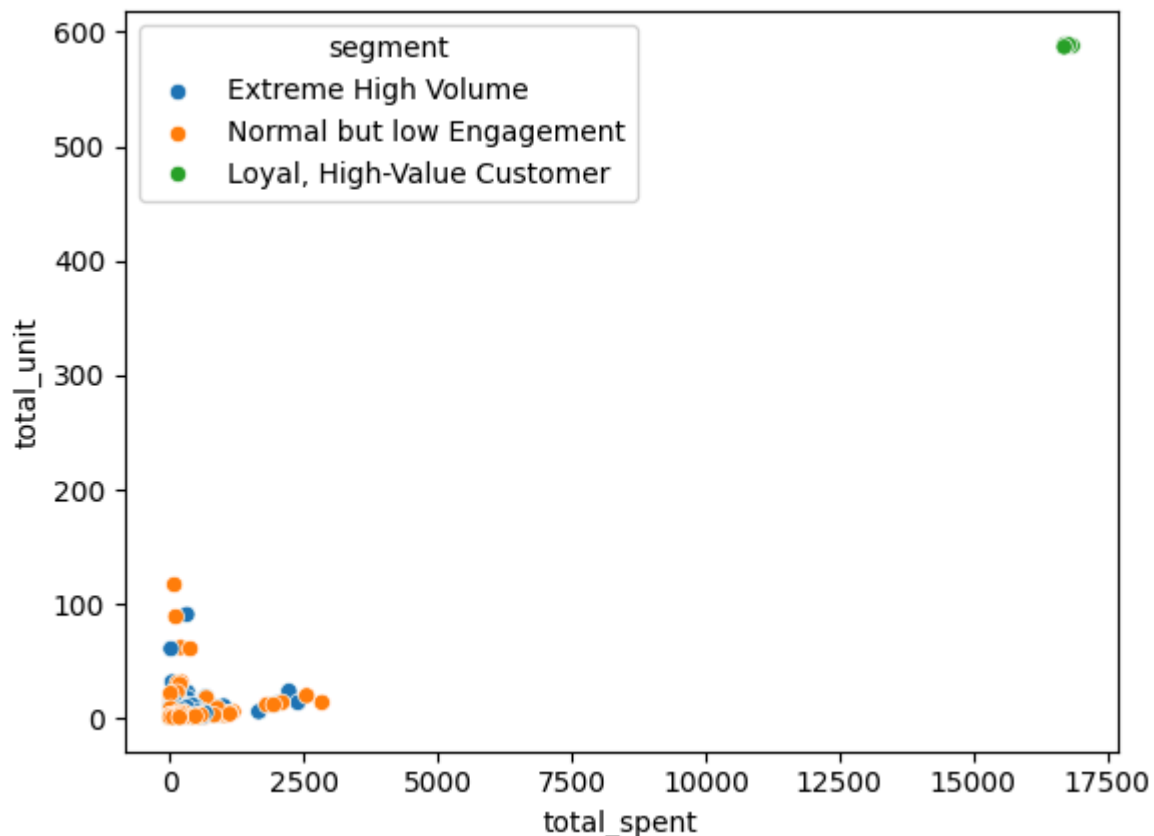
```
Out[19]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th>
<th>customer_id</th> <th>registered_to_order_duration</th> <th>first_to_last_order_duration</th>
<th>total_unit</th> <th>total_spent</th> <th>cluster</th> <th>segment</th> </thead> <tbody>
<th>707</th> <th>746</th> <th>767</th> <th>773</th> <th>804</th> </tbody> <tbody>
</tbody>
```

CUST_00025	-14	296	4.0	284.169166	2	Extreme High Volume
CUST_00365	-22	307	4.0	27.578441	2	Extreme High Volume
CUST_00268	-11	169	3.0	27.539251	2	Extreme High Volume
CUST_00518	-6	272	5.0	13.548483	2	Extreme High Volume
CUST_00458	-8	0	1.0	80.609741	0	Normal but low Engagement

```
In [20]: # visualize the data
```

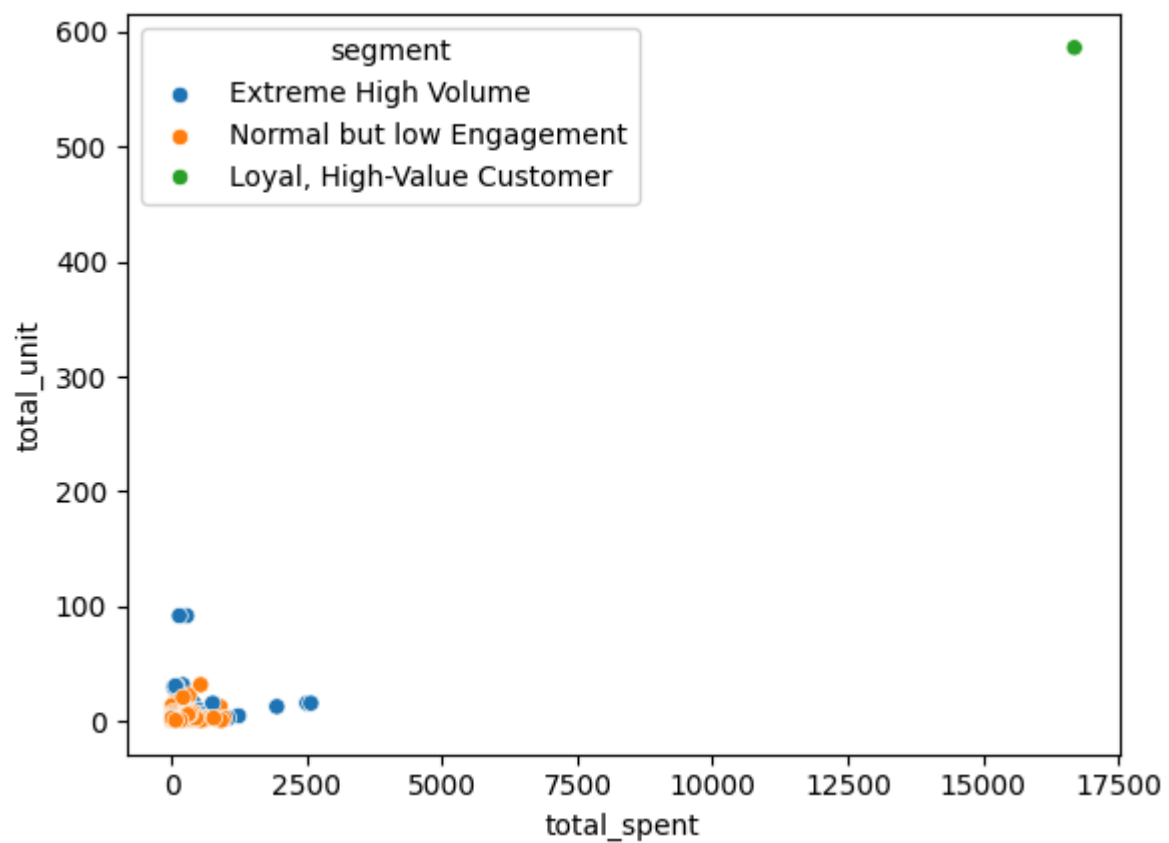
```
sns.scatterplot(
    data = df2_normal,
    x= 'total_spent',
    y= 'total_unit',
    hue= 'segment',
    palette= 'tab10'
)
```

```
Out[20]: <Axes: xlabel='total_spent', ylabel='total_unit'>
```



```
In [21]: sns.scatterplot(
    data = df2_pre_registration,
    x= 'total_spent',
    y= 'total_unit',
    hue= 'segment',
    palette= 'tab10'
)
```

```
Out[21]: <Axes: xlabel='total_spent', ylabel='total_unit'>
```



In [22]: `# combine the data`

```
df2_normal['segment_type'] = 'normal'
df2_pre_registration['segment_type'] = 'pre_registration'
```

In [23]: `df2_normal.head()`

Out[23]: `<style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th>
<th>customer_id</th> <th>registered_to_order_duration</th> <th>first_to_last_order_duration</th>
<th>total_unit</th> <th>total_spent</th> <th>cluster</th> <th>segment</th>
<th>segment_type</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th> <th>3</th>
<th>4</th> </tbody> <tbody> </tbody>`

CUST_03731	562	692	6.0	295.254288	2	Extreme High Volume	normal
CUST_03071	466	272	3.0	254.375567	2	Extreme High Volume	normal
CUST_00908	263	0	1.0	97.368058	0	Normal but low Engagement	normal
CUST_02852	417	542	2.0	88.990766	2	Extreme High Volume	normal
CUST_00510	173	406	2.0	257.549943	2	Extreme High Volume	normal

In [32]: `df2_pre_registration.head()`

```
Out[32]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th {
vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th>
<th>customer_id</th> <th>register_to_order</th> <th>first_to_last_order</th> <th>total_unit</th>
<th>total_spent</th> <th>cluster</th> <th>segment</th> <th>segment_type</th> </thead>
<tbody> <th>4</th> <th>5</th> <th>7</th> <th>8</th> <th>11</th> </tbody> <tbody></tbody>
C215074L  -65  509  17.0  28298942.40  1  Loyal, High-Value Customer  pre_registration
C483469L   -5  297  16.0  16317314.00  0  Normal but low Engagement  pre_registration
C545190L   -8  129  15.0   3965170.00  0  Normal but low Engagement  pre_registration
C492850L  -29  450  34.0  48588804.00  2           Extreme High Volume  pre_registration
C869172L   -7  267  57.0   4553254.62  1  Loyal, High-Value Customer  pre_registration
```

```
In [24]: df2_segmented = pd.concat([df2_normal,df2_pre_registration])
```

```
In [25]: df2_segmented.tail(20)
```

```
Out[25]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th {
vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th>
<th>customer_id</th> <th>registered_to_order_duration</th> <th>first_to_last_order_duration</th>
<th>total_unit</th> <th>total_spent</th> <th>cluster</th> <th>segment</th>
<th>segment_type</th> </thead> <tbody> <th>3676</th> <th>3677</th> <th>3678</th>
<th>3679</th> <th>3680</th> <th>3681</th> <th>3682</th> <th>3683</th> <th>3911</th>
<th>3913</th> <th>3915</th> <th>3916</th> <th>3917</th> <th>3918</th> <th>3919</th>
<th>3920</th> <th>3921</th> <th>3922</th> <th>3923</th> <th>3924</th> </tbody> <tbody>
</tbody>
```

CUST_01234	-437	0	1.0	183.630465	0	Normal but low Engagement	pre_registration
CUST_02347	-330	0	1.0	166.368279	0	Normal but low Engagement	pre_registration
CUST_00860	-465	0	1.0	1.716888	0	Normal but low Engagement	pre_registration
CUST_01780	-375	0	1.0	1.026400	0	Normal but low Engagement	pre_registration
CUST_01819	-370	0	1.0	2.398044	0	Normal but low Engagement	pre_registration
CUST_03585	-187	0	1.0	2.985892	0	Normal but low Engagement	pre_registration
CUST_01814	-393	0	1.0	12.223494	0	Normal but low Engagement	pre_registration
CUST_02643	-297	0	1.0	15.769240	0	Normal but low Engagement	pre_registration
CUST_03862	-20	0	1.0	208.743679	0	Normal but low Engagement	pre_registration
CUST_03908	-28	5	7.0	343.347670	0	Normal but low Engagement	pre_registration
CUST_03933	-6	116	4.0	10.002737	2	Extreme High Volume	pre_registration
CUST_03784	-33	0	1.0	2.239419	0	Normal but low Engagement	pre_registration
CUST_03923	-28	0	1.0	1.866182	0	Normal but low Engagement	pre_registration
CUST_03969	-16	0	6.0	298.570494	0	Normal but low Engagement	pre_registration
CUST_03916	-66	0	1.0	14.929458	0	Normal but low Engagement	pre_registration
CUST_03971	-56	0	1.0	158.623623	0	Normal but low Engagement	pre_registration
CUST_03857	-71	0	1.0	3.359128	0	Normal but low Engagement	pre_registration
CUST_03802	-109	0	1.0	60.650922	0	Normal but low Engagement	pre_registration
CUST_03817	-109	0	3.0	4.198910	0	Normal but low Engagement	pre_registration
CUST_03928	-116	0	1.0	71.185521	0	Normal but low Engagement	pre_registration

```
In [26]: # save the data

df2_segmented.to_excel(r"C:\ecommerce_powerBi_sql\customer_segment.xlsx" , index = False)

In [ ]:
```